SENTINEL · DESIGN REFERENCE

Sentinel

A Telegram bot for social-media sentiment, pre/post-market price moves, and US macroeconomic releases — with the “why” attached.

One chat interface over a streaming data platform. Kafka ingests news and forum chatter; Spark Structured Streaming scores every message with FinBERT and lands it in a Delta Lake lakehouse; a point-in-time-safe alignment turns sentiment into a leading indicator that is backtested before it is ever trusted. On top of that sit three push features: a **sentiment leading-indicator alert**, a **pre/post-market price-move explainer**, and a **US macro-release explainer** — each one grounded in real data and each one honest about what it does not know.

Telegram Bot API · Apache Kafka · PySpark Structured Streaming · Delta Lake · FinBERT · transformers · Ray · vectorbt · yfinance · Alpaca · FRED · BLS · Finnhub news · LLM explainer (Claude API) · Streamlit · Docker Compose

Document — Technical blueprint & delivery plan

Interface — Telegram bot (commands + push alerts)

Repository — stock-sentiment-analysis-pipeline

Features — 3 (sentiment · price-move · macro)

Status — Feature 1 largely built; 2 & 3 specified

Sections — 01–17

Written from the code that exists in the repository, not from aspiration: every architectural claim about the sentiment pipeline maps to a module under `ingestion/`, `data_pipeline/`, `ml_engine/`, `quant_backtest/`, and `telegram_bot/`. Work that is designed but not yet built is labelled TO BUILD rather than described as if it shipped. Figures for third-party quotas and pricing were reasonable at time of writing and must be re-verified before launch.

00 Contents

BLUEPRINT MAP

HOW TO READ THIS

The three product features map to three delivery tracks in §13. Feature 1 (sentiment) is the streaming backbone and is largely implemented today; Features 2 (price-move alerts) and 3 (macro alerts) reuse that backbone and are specified here as new work. Sections 01–12 describe the whole system; §13 is where each feature becomes a sequence of independently verifiable milestones; §16–17 cover the parts a reviewer will scrutinise hardest — the “not financial advice” guardrails and the retrieval-grounded explainer.

01 Executive Summary & System Architecture

BRIEF §1–2

1.1 Project goal

A trader opens one thing: a Telegram chat. Behind it, a streaming data platform is doing three jobs at once.

It **reads the room** — ingesting forum posts and news wires, scoring each one with a financial-domain language model, and pinging the user the moment aggregated sentiment on a ticker they follow crosses a threshold, *before* the move shows up cleanly in price. It **watches the tape** — when a subscribed ticker gaps in pre-market or post-market trading, it doesn't just say “NVDA −6%”, it answers *why*, grounded in the headlines that actually broke. And it **watches Washington** — when the Fed, BLS, or BEA releases a number that moves markets, it explains what printed, how it compares to consensus, and why it matters.

The design target: **zero dashboards to babysit**. A passive Streamlit dashboard exists for the builder; the trader lives entirely in push notifications and a handful of slash commands.

1.2 Architecture in one paragraph

Ingestion producers publish ticker-tagged text to **Kafka**. A **Spark Structured Streaming** job consumes each micro-batch, writes it verbatim to a **Delta Lake** raw table for a full audit trail, scores it with **FinBERT** through a pandas-UDF that loads the model once per executor, and merges the scored rows into a second Delta table keyed on message id. A separate alignment step pulls OHLCV bars (yfinance or Alpaca), aggregates sentiment into fixed windows, and joins them **point-in-time** — a bar at time `t` may only see sentiment strictly before `t`, which is the single guard that keeps the **vectorbt** backtest honest. The **Telegram bot** and the Streamlit dashboard both read those Delta tables through one cached data-service layer. Two new market-event services — a price monitor and a macro monitor — publish structured alerts to their own Kafka topics; a dispatcher fans them out to subscribed chats, attaching a retrieval-grounded explanation from a hosted **LLM**. Everything runs as containers under one Docker Compose file.

1.3 Key design decisions

DECISION	RATIONALE	WHAT IT COSTS YOU
Kafka + Spark + Delta, not a single Python script	The workload is genuinely a stream: unbounded, bursty, replayable. Kafka gives durable buffering and back-pressure; Spark gives exactly-once micro-batch semantics; Delta gives ACID upserts and time-travel so a replayed partition never double-counts. This is the honest shape of the problem, not resume-driven design.	Heavy footprint: a JVM, Zookeeper, a broker. Overkill at 4 tickers; the right call the moment the universe or the source count grows. Named in §14 as the first thing to right-size *down* for a tiny demo.
FinBERT as the “own ML pipeline”, an LLM only for explanation	Sentiment scoring is a high-volume, low-latency classification job — a fine-tuned domain model (`yiyanghkust/finbert-tone`) is faster, cheaper, and more reproducible than an LLM call per post. The LLM is reserved for the low-volume, high-value job of *explaining* a price or macro move from retrieved sources.	Two model stacks to operate. But they fail independently and scale independently, which is the point.
Point-in-time alignment, always	`merge_asof(direction="backward", allow_exact_matches=False)` in `data_pipeline/data_alignment.py` . A backtest that peeks at future sentiment is worse than no backtest — it manufactures confidence. This guard is load-bearing for the whole “leading indicator” claim.	A sliver of signal is discarded at each bar boundary. Correct trade, every time.
Explanations are retrieval-grounded, never free-form	The LLM is handed the actual headlines / the actual release numbers and instructed to explain only from them, cite them, and refuse when sources are thin. A bot that invents a reason a stock moved is worse than one that says “I don't have enough to say.” See §16 GUARD-1.	Occasionally a terse “no clear catalyst in the news yet” instead of a satisfying story. That is the honest answer.
Sentiment as a leading indicator, proven by backtest — not asserted	The crossing signal is run through vectorbt against buy-and-hold before it is shipped. If it doesn't beat the benchmark out-of-sample, that is information, and the bot reports the backtest stats rather than hiding them.	The headline feature must survive contact with its own P&L curve. Deliberate.
One state store for the bot: a locked JSON file	`telegram_bot/state.py` . Chat count is small; writes are rare (only on preference changes and alert de-dup). No extra database service to run for a demo.	Single-instance only, and not the answer past a few thousand chats. Migration path to Postgres/Redis in §14.
Not financial advice, structurally	The bot reports data and explains events; it never recommends a trade. This is enforced by what the code can and cannot emit (§16), not only by a footer disclaimer.	Slightly less “actionable”-sounding copy. The only defensible choice.

2.1 High-level architecture

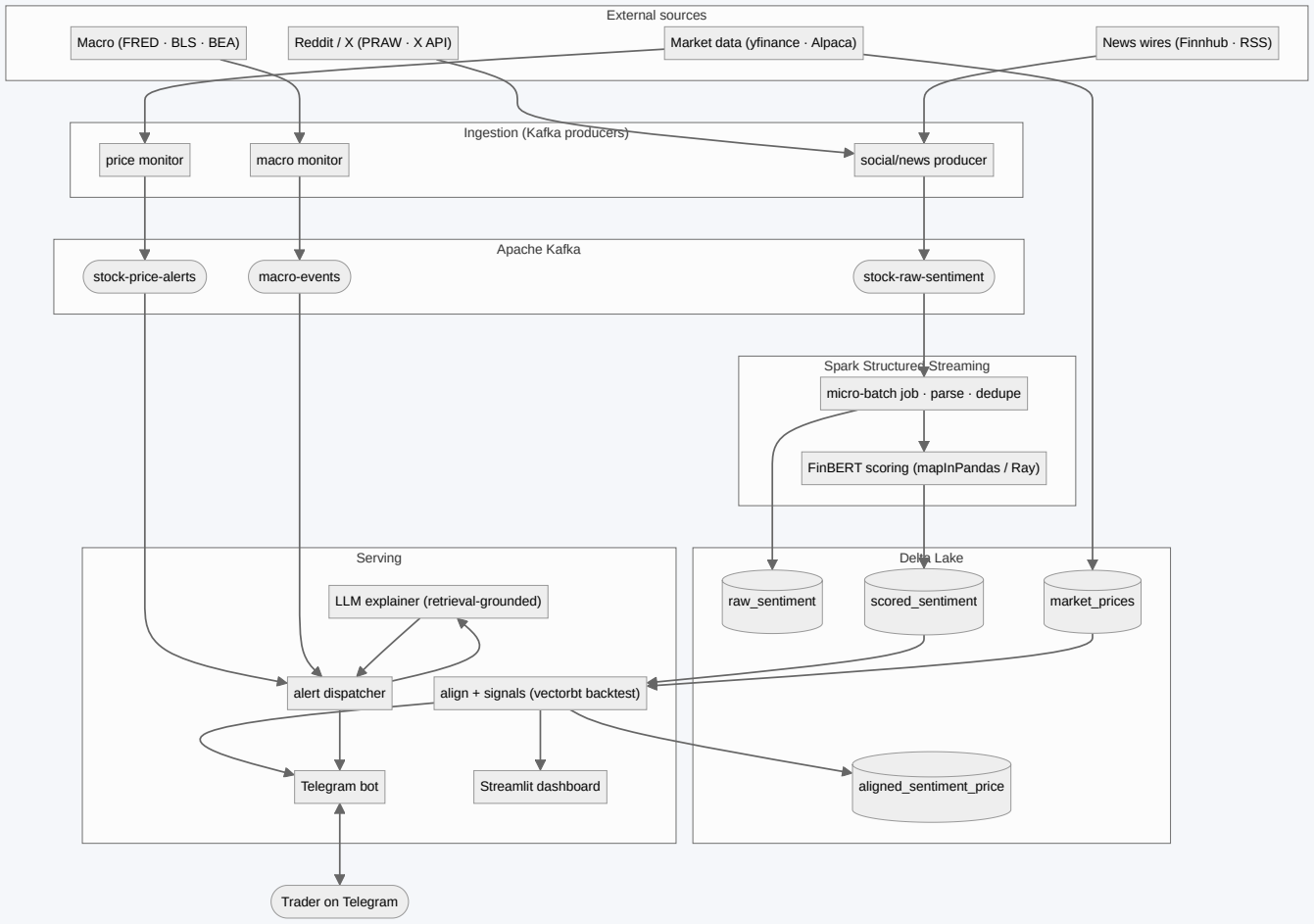

Fig 1 — Kafka buffers, Spark scores, Delta stores, the bot serves. Solid backbone (sentiment) is built; the price/macro monitors, dispatcher, and explainer are the new work in Features 2–3.

2.2 Components

COMPONENT	RESPONSIBILITY	MODULE	STATUS
Social/news producer	Pull **real** posts — RSS (keyless) + Reddit/Finnhub (keyed) — and publish to Kafka. No synthetic data.	`ingestion/kafka_producer.py`, `ingestion/sources/`	BUILT
Kafka utilities	Topic creation, JSON (de)serialization, schema validation, tuned producer/consumer builders	`ingestion/kafka_utils.py`	BUILT
Streaming scorer	Kafka → parse → dedupe → raw Delta → FinBERT → scored Delta, in `foreachBatch`	`data_pipeline/spark_streaming.py`	BUILT
Delta writer	ACID MERGE upserts keyed on id / (ticker,timestamp); partitioning; OPTIMIZE/ZORDER compaction	`data_pipeline/delta_writer.py`	BUILT
Alignment	Fetch OHLCV; window-aggregate sentiment; point-in-time `merge_asof`	`data_pipeline/data_alignment.py`	BUILT
FinBERT model	Device-aware tokenizer+model wrapper; batch inference; process-local singleton	`ml_engine/finbert_model.py`	BUILT
Distributed inference	Ray actor pool + Spark pandas-UDF scoring path	`ml_engine/distributed_inference.py`	BUILT
Signal generator	Rolling sentiment features; entry/exit *crossing* signals	`quant_backtest/signal_generator.py`	BUILT
Backtester	vectorbt strategy vs. buy-and-hold; return/Sharpe/drawdown	`quant_backtest/backtester.py`	BUILT
Telegram bot	Commands, inline keyboards, callback queries, push alert loop	`telegram_bot/`	BUILT
Bot data service	TTL-cached Delta/align/signal/backtest access shared by all handlers	`telegram_bot/data_service.py`	BUILT
Price monitor	Extended-hours quote stream → % move / z-score → `stock-price-alerts`	`ingestion/price_monitor.py`	TO BUILD
Macro monitor	Release-calendar poll → actual-vs-consensus → `macro-events`	`ingestion/macro_monitor.py`	TO BUILD

COMPONENT	RESPONSIBILITY	MODULE	STATUS
LLM explainer	Retrieve headlines/release → grounded, cited explanation, or refuse	`explain/`	TO BUILD
Alert dispatcher	Consume alert topics → look up subscribers → send with explanation	`telegram_bot/dispatcher.py`	TO BUILD
Dashboard	Price+sentiment chart, backtest chart, recent-posts table (builder-facing)	`dashboard/app.py`	BUILT

2.3 Request lifecycle — a sentiment micro-batch

```
# One 30-second Spark micro-batch, from a forum post to a possible alert
1.  producer publishes {id, ticker, timestamp, source, text} to stock-raw-sentiment
2.  Spark readStream pulls the batch; from_json → typed rows; to_timestamp(timestamp)
3.  process_batch(): dropDuplicates(["id"]).cache(); count()          # idempotent on replay
4.  write_raw_sentiment(df)         → append to raw_sentiment Delta (audit trail, partition date/ticker)
5.  score_dataframe_with_spark(df)  → mapInPandas: FinBERT loaded ONCE per partition, batched
6.  upsert_scored_sentiment(df)     → MERGE into scored_sentiment on id (no dupes ever)
    ———————— streaming job ends here; serving reads Delta on demand ————————
7.  bot/dashboard: read scored_sentiment + fetch OHLCV
8.  aggregate_sentiment(window) → merge_asof backward, exact=False   # point-in-time, no lookahead
9.  add_rolling_sentiment() → generate_signals(): crossing above ENTRY / below EXIT
10. run_backtest() vs buy-and-hold                                   # is this signal actually worth it?
11. subscribed? alert loop sees a fresh crossing on the latest bar → push to chat, de-dup on bar timestamp
```

Steps 1–6 are the durable, exactly-once path. Everything after is stateless read-and-render, so a crash while serving costs a retry, never a lost message — Kafka + Delta already hold the truth.

2.4 Data flow notes

Raw before scored. Every message is persisted verbatim to `raw_sentiment` before FinBERT runs. If the model is swapped or a bug is found, the entire corpus can be re-scored from the audit table without touching Kafka.

Sentiment aggregated at the window's *close*. `aggregate_sentiment` shifts each resample bucket to its right edge, so a window's aggregate carries the timestamp at which its data was fully observed — the value later used as the point-in-time cutoff. This is subtle and correct; getting it wrong silently reintroduces lookahead.

Prices fetched, not streamed, for backtest. Research-grade OHLCV comes from yfinance/Alpaca in batch and is small enough to align in Pandas. Live extended-hours quotes for Feature 2 are a separate, streaming concern (§04, §07).

2.5 Deployment shape

One Docker Compose project. Today: `zookeeper`, `kafka`, `kafka-producer`, `ml-inference-spark`, `telegram-bot`. Features 2–3 add `price-monitor`, `macro-monitor`, and `alert-dispatcher` services, plus an optional `dashboard`. The Spark container carries a CPU/memory reservation because FinBERT inference is the hot path; everything else is light.

02 Technology Stack

BRIEF §3

Cost key: FREE open-source / unmetered · FREE TIER free within a quota · PAID costs money. Every entry: purpose, why, cost, alternatives, limits.

Streaming & data platform

Apache Kafka FREE

Purpose: durable ingestion bus for all inbound text and market/macro events.
Why: replayable, partitioned by ticker (ordering per key), decouples producers from the Spark consumer. `confluent-kafka` (librdkafka) for producers, Spark's built-in source for consumption.
Alts: Redpanda (Kafka-API, lighter, no ZK), AWS Kinesis (managed, paid), a plain Redis stream (fine at demo scale).
Limits: operationally heavy — ZK/broker/JVM. Single-broker, RF=1 in the compose file: no real durability, fine for dev.

PySpark 3.5 · Structured Streaming FREE

Purpose: the scoring stream and all batch transforms.
Why: exactly-once micro-batch with checkpointing; `mapInPandas` lets FinBERT run once per partition instead of once per row — the thing that makes model inference tractable in a stream.
Alts: Flink (true streaming, steeper), Bytewax/Faust (Python-native, smaller ecosystem).
Limits: JVM startup + shuffle overhead; needs a JDK in the image (Dockerfile installs OpenJDK 17). `mapInPandas` is unsupported on a streaming DF directly — scoring lives inside `foreachBatch`.

Delta Lake 3.1 FREE

Purpose: the lakehouse — raw, scored, prices, aligned tables.
Why: ACID `MERGE` gives idempotent upserts (replay-safe), time-travel gives reproducible backtests, partitioning + ZORDER keeps small-file bloat down under frequent micro-batch appends.
Alts: Apache Iceberg, Hudi (comparable), plain Parquet (no ACID, no upsert), Postgres/TimescaleDB (great for serving, weaker for a replayable audit lake).
Limits: small-file problem from streaming appends — `optimize_table()` exists for exactly this.

Ray 2.31 FREE

Purpose: the ad-hoc/batch FinBERT inference path (`RayInferencePool`), each actor holding one loaded model.
Why: clean actor model for a pool of GPU/CPU inference workers outside Spark (e.g. backfills, notebooks).
Alts: a multiprocessing pool, Spark alone (already used for the stream), a dedicated Triton/TorchServe endpoint.
Limits: two inference paths (Ray + Spark UDF) is a small redundancy — the Spark path is the production one; keep Ray for batch.

Machine learning

FinBERT · `transformers` · `torch` FREE

Purpose: the "own ML pipeline" — financial-tone sentiment on every post.
Why: `yiyanghkust/finbert-tone` is fine-tuned on financial text; net score = P(pos) − P(neg) ∈ [−1,1]. Domain-specific and cheap per inference, unlike an LLM call per message.
Alts: `ProsusAI/finbert`, a distilled/ONNX-quantised head for speed, a fine-tuned DistilBERT of your own.
Limits: 128-token cap (long posts truncated); tuned for headlines/news register, weaker on heavy slang/emoji (a real WSB gap, §16); CPU inference is the throughput ceiling — batch aggressively.

Hosted LLM — Anthropic Claude API FREE TIER / PAID

Purpose: the *explanation* layer only — turn retrieved headlines / a macro release into a short, cited, plain-English "why".
Why: strong at grounded summarisation and at refusing when sources are thin; a Haiku-class tier is cheap enough for low-volume event explanations, a Sonnet-class tier handles the harder macro reasoning.
Alts: Gemini Flash (generous free tier), GPT-4o-mini, a local Llama/Qwen-Instruct (free, heavier to host). Kept behind an `LLMProvider` interface so the choice is one config flip.
Limits: paid per token beyond any free allowance; must be rate-limited and cached; never in the hot per-post path — events only. Prompts must carry no user PII.

Market, news & macro data

LIBRARY / API	PURPOSE	COST	LIMITS WORTH KNOWING
`yfinance`	Research OHLCV, default provider for alignment/backtest	FREE	Unofficial, rate-limited, occasional gaps; `prepost=True` extended-hours data is unreliable for real-time alerting.
`alpaca-py`	Cleaner intraday bars; **real-time extended-hours quotes** for Feature 2	FREE TIER	Free with a paper account; IEX feed on free tier (not full SIP). Websocket for live pre/post-market. Keys already in `.env.example`.
Finnhub	Per-symbol **company news** (the "why" source) + economic calendar	FREE TIER	~60 req/min free; company-news endpoint is the primary retrieval source for the price-move explainer.
FRED (St. Louis Fed)	Official macro series: Fed funds, CPI, PCE, unemployment	FREE	Free API key; release calendar via FRED/ALFRED. The authoritative macro source.
BLS / BEA	CPI, employment (BLS); GDP, PCE (BEA)	FREE	Free keys; useful for consensus-vs-actual context on the big prints.

Interface, quant & ops

python-telegram-bot 21 FREE

Why: mature async bot framework; built-in `JobQueue` powers the alert poll loop; inline keyboards for ticker/window pickers.
Limits: polling mode today (simple); a webhook + FastAPI is the scale path. Telegram rate limit ≈30 msg/s global, ~1 msg/s per chat — matters for broadcast fan-out (§07).

vectorbt FREE

Why: vectorised backtests; `Portfolio.from_signals` vs `from_holding` gives strategy-vs-benchmark in a few lines.
Limits: in-memory; fine for a handful of tickers. Sharpe/return are sensitive to bar frequency — always pass an explicit `freq`.

Streamlit · Plotly · kaleido FREE

Why: builder-facing dashboard; `kaleido` rasterises Plotly figures to PNG so the bot can send the same charts as photos.
Limits: kaleido needs a headless-Chromium-ish renderer in the image; charts are static in chat (Telegram has no interactive plots).

Docker Compose FREE

Why: one file brings up ZK, Kafka, producer, Spark scorer, bot. Healthchecks gate start order.
Limits: single host; not an orchestrator. K8s is the scale-out story, deferred (§14).

`python-dotenv` + `config/settings.py` FREE

Why: one settings module, every value overridable by env var, so identical code runs on a laptop or in compose.
Limits: no typed validation yet — a `pydantic-settings` pass is a cheap hardening win (§10).

pandas · numpy · scipy · pyarrow FREE

Why: the alignment/signal/backtest math and Arrow interchange between Spark and Pandas.
Limits: Pandas alignment assumes the research set fits in memory — true for a small universe, revisit for hundreds of tickers.

Deliberately not used (yet)

NOT USED	WHY
A vector DB	Retrieval for the explainer is a per-ticker recent-headlines fetch, not semantic search over a corpus. Add one only if a knowledge base appears (§14).
A relational DB for bot state	A locked JSON file covers a few hundred chats. Postgres/Redis is the documented upgrade the moment fan-out or multi-instance is real.
Airflow / Dagster	Spark Structured Streaming + the bot's `JobQueue` cover scheduling. An orchestrator is warranted once backfills and retrains multiply.
A custom sentiment model trained from scratch	FinBERT is a strong, free starting point. Fine-tuning on labelled cashtag data is a §14 improvement, not a day-one requirement.
Kubernetes	Compose is enough for one host. Named as the scale-out path, not built.

03 Telegram Bot Channel

BRIEF §2 · INTERFACE

The bot is the entire product surface. It does two things a passive dashboard cannot: it takes commands, and it *pushes* — the moment a signal fires, a price gaps, or a macro number prints.

3.1 Setup

1. Create the bot with **@BotFather**, obtain the token, set `TELEGRAM_BOT_TOKEN` (see `.env.example`). The compose file refuses to start `telegram-bot` without it.
2. Register the command list with BotFather (`/setcommands`) so Telegram shows the menu.
3. Run `python -m telegram_bot.bot` — `build_application()` wires handlers and starts **long-polling** (`run_polling(allowed_updates=["message","callback_query"])`).

> **POLLING VS WEBHOOK**
>
> Polling needs no public URL and is perfect for a demo. The scale path is a Telegram **webhook** into a small FastAPI endpoint (validate the secret token, enqueue, return 200 fast) — the same write-then-process shape used everywhere else. Called out in §14; not needed to ship.

3.2 Command surface (as built)

COMMAND	DOES	HANDLER
`/start`, `/help`	Welcome + command reference (HTML)	`start_command` / `help_command`
`/ticker [SYM]`	Set active ticker; no arg → inline keyboard of the known universe	`ticker_command`
`/window [15m|1h|4h|24h]`	Set the rolling-sentiment window; no arg → inline keyboard	`window_command`
`/price`	PNG: price vs. net-sentiment dual-axis chart for the active ticker	`price_command`
`/backtest`	PNG + stats: sentiment strategy vs. buy-and-hold (return, Sharpe, drawdown)	`backtest_command`
`/posts`	Recent FinBERT-scored posts, colour-emoji by score	`posts_command`
`/subscribe`, `/unsubscribe`	Toggle push alerts on entry/exit crossings for the active ticker	`subscribe_command` / `unsubscribe_command`
`/status`	Show this chat's ticker, window, alert state	`status_command`
`/refresh`	Clear the data-service cache; next command re-reads Delta / yfinance	`refresh_command`
inline buttons	Ticker/window taps → update state, re-render chart	`callback_query_handler`

Commands added for Features 2–3 (§07): `/watch SYM` and `/unwatch SYM` (per-ticker price-move subscriptions), `/macro on|off` (macro broadcast opt-in), and `/why` (explain the active ticker's latest move on demand).

3.3 Per-chat state

`telegram_bot/state.py` holds a `ChatState` per chat — `ticker`, `window`, `subscribed`, and a `last_alert` map (`ticker → last-alerted bar timestamp`) that guarantees a given crossing is never announced twice. Persisted to a lock-guarded JSON file so preferences survive restarts with no database service. `subscribed_chat_ids()` is the fan-out list the alert loop iterates.

Feature 2/3 extend this with `watched_tickers: set[str]` and `macro_opt_in: bool`, plus a per-event de-dup key so a re-polled release or a re-observed gap fires once.

3.4 Inbound → response flow

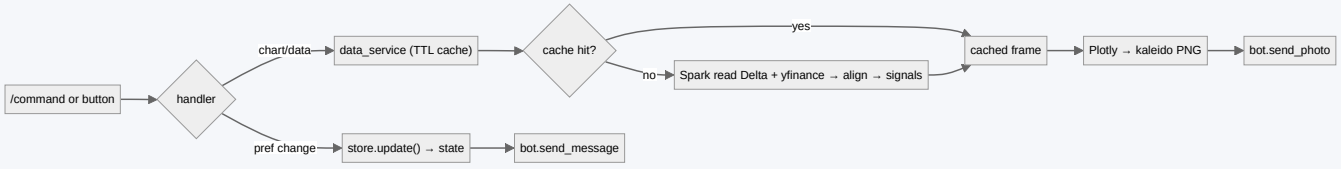

Fig 2 — A TTL cache (`data_service._TTLCache`) stops a burst of commands from re-triggering a Spark read or a yfinance call per keystroke.

3.5 Push alerts

The one thing a dashboard can't do. `telegram_bot/alerts.py::check_and_send_alerts` runs on the `JobQueue` every `TELEGRAM_ALERT_POLL_SECONDS` (default 60s). For each subscribed chat it loads the latest bar for that chat's ticker/window; if it's a fresh entry or exit crossing (not equal to `last_alert[ticker]`), it pushes a message and records the bar timestamp. One chat's failure is caught and logged so it can't kill the loop for everyone.

Feature 2/3 alerts arrive differently — pushed by the **dispatcher** off Kafka topics rather than polled off Delta — but converge on the same `send_message` and the same per-event de-dup discipline (§07, §09).

3.6 Message formatting rules

- **HTML parse mode**, and every user/source string is `html.escape()`-d (see `posts_command`) — a post body containing `<` must never break the message or inject markup.
- Charts go as **photos** (PNG via kaleido), captioned; stats ride in the caption with `parse_mode="HTML"`.
- Tickers rendered as `$SYM`; sentiment as 🟢/🟡/🔴 by the same ±0.2 thresholds the dashboard uses, so the two surfaces read identically.
- Every alert names the **bar timestamp** it fired on — an alert without a “when” is useless in markets.

3.7 Local development

No phone needed for most work: unit-test handlers against a fake `Update`/`Context`, and drive the data path directly through `data_service`. For live testing, one real bot token and a DM to the bot exercises the whole loop; `docker compose up` brings up the producer + Spark scorer so real scored data exists to chart.

04 Data Sources & Ingestion

BRIEF §3 · FEEDS

Four feed families. Social/news ingestion is now **real** (no synthetic generator); the remaining work is the two market-event monitors and wiring the keyed sources.

4.1 Social & news (Feature 1 input)

`ingestion/kafka_producer.py` pulls from **real sources only**, via the `ingestion/sources/` package: keyless **RSS** (Google News per ticker — the default, no credentials), plus **Reddit** (PRAW) and **Finnhub** company news when their keys are set (`INGEST_SOURCES` selects which are active). Every source emits the same contract everything downstream depends on:

```
{"id": uuid, "ticker": "NVDA", "timestamp": ISO-8601-UTC, "source": "reddit|news|twitter", "text": "..."}
```

Each source implements one `fetch() → list[Post]` method behind the `Source` interface (`ingestion/sources/base.py`); adding a source changes no downstream line:

SOURCE	CLIENT	APPROACH	COST / LIMIT
Reddit (r/wallstreetbets, r/stocks, r/investing)	`PRAW`	Stream new submissions/comments; extract `$CASHTAG` s and known tickers; publish one message per (post × ticker).	FREE OAuth app; ~60 req/min.
X / Twitter	X API v2 (filtered stream / recent search)	Rule per cashtag; publish matching tweets. Honest caveat: the usable X tier is now paid and metered.	PAID — the real budget line for social; Reddit alone is a viable v1.
News wires	Finnhub company-news / RSS	Poll per-ticker headlines; publish title+summary. Doubles as the retrieval source for Feature 2.	FREE TIER .

Ticker extraction is the one bit of real NLP at ingestion: a cashtag regex (`\$[A-Z]{1,5}`) plus an allow-list from `settings.TICKERS` , guarding against false positives like `$1` or common words. A post mentioning two tickers becomes two messages (Kafka key = ticker, preserving per-ticker order).

4.2 Market data (Features 1 & 2)

`data_pipeline/data_alignment.py::fetch_market_data` already abstracts provider choice (`yfinance` default, `alpaca` opt-in) behind one long-format OHLCV frame. Two distinct needs:

- **Research bars** (backtest/align): batch pull, 60d × 15m, either provider. Built.
- **Live extended-hours quotes** (Feature 2 alerting): Alpaca's real-time websocket, subscribed to watched tickers, delivering pre-market (04:00–09:30 ET) and post-market (16:00–20:00 ET) trades/quotes. New — `ingestion/price_monitor.py` .

4.3 Macro (Feature 3)

Two parts: **when** a release drops, and **what** it said.

NEED	SOURCE	NOTES
Release calendar (when to look)	FRED release calendar / Finnhub economic calendar	Poll a few times an hour; know that CPI, FOMC, NFP, PCE, GDP are due, with expected timestamps.
Actual value	FRED series (authoritative), BLS/BEA	Fetch the just-released observation for the series (e.g. `CPIAUCSL` , `FEDFUNDS` , `UNRATE`).
Consensus / prior	Economic-calendar consensus fields; prior from FRED history	“Actual vs. expected vs. prior” is the whole story a macro alert needs to carry.

Macro is **market-wide**, not per-ticker: a single event fans out to every `macro_opt_in` chat, so the monitor publishes one `macro-events` record per release, not per subscriber.

4.4 Ingestion guarantees

- **Idempotency by construction.** Every message carries a stable `id` ; Spark `dropDuplicates(["id"])` and Delta `MERGE` mean a replayed Kafka partition never double-counts. Price/macro events carry a natural key (`ticker+bar` , `series+release_time`) for the same reason.
- **Validation at the edge.** `validate_raw_sentiment` checks required fields before anything trusts a payload.
- **Back-pressure is free.** If Spark falls behind, Kafka retains; nothing is dropped, throughput just lags — the correct failure mode for bursty market opens.
- **Tuned producer.** `acks=all` , retries, `linger.ms=50` , snappy compression (`build_producer`) — durability over microseconds, right for this data.

05 AI / ML Processing Pipelines

BRIEF §3 · PIPELINES

Five pipelines, one shape: **ingest → transform → model → store → serve**. Only one of them (sentiment) runs a model on every record; the two event explainers call an LLM sparingly, on events, grounded in retrieved data.

5.1 Sentiment scoring (streaming) — built

Kafka → Spark micro-batch → FinBERT (`mapInPandas`) → Delta. The model loads once per partition, not per row; net score = P(pos) − P(neg). Raw rows are persisted before scoring for a full re-score capability. Detailed in §06.

Degraded mode: if the model fails to load, the raw table still fills, so nothing is lost and a backfill re-scores once the model is back. The stream is designed so scoring is the only thing that can fail, and it fails safe.

5.2 Signal & leading-indicator pipeline — built

Aligned sentiment+price → rolling means (`add_rolling_sentiment`) → crossing signals (`generate_signals`) → vectorbt backtest. Crossings (not levels) are used so the backtester sees discrete entry/exit events. The “leading indicator” claim is *tested* here, not asserted (§06.4, §07.1).

5.3 Price-move detection & explanation — to build

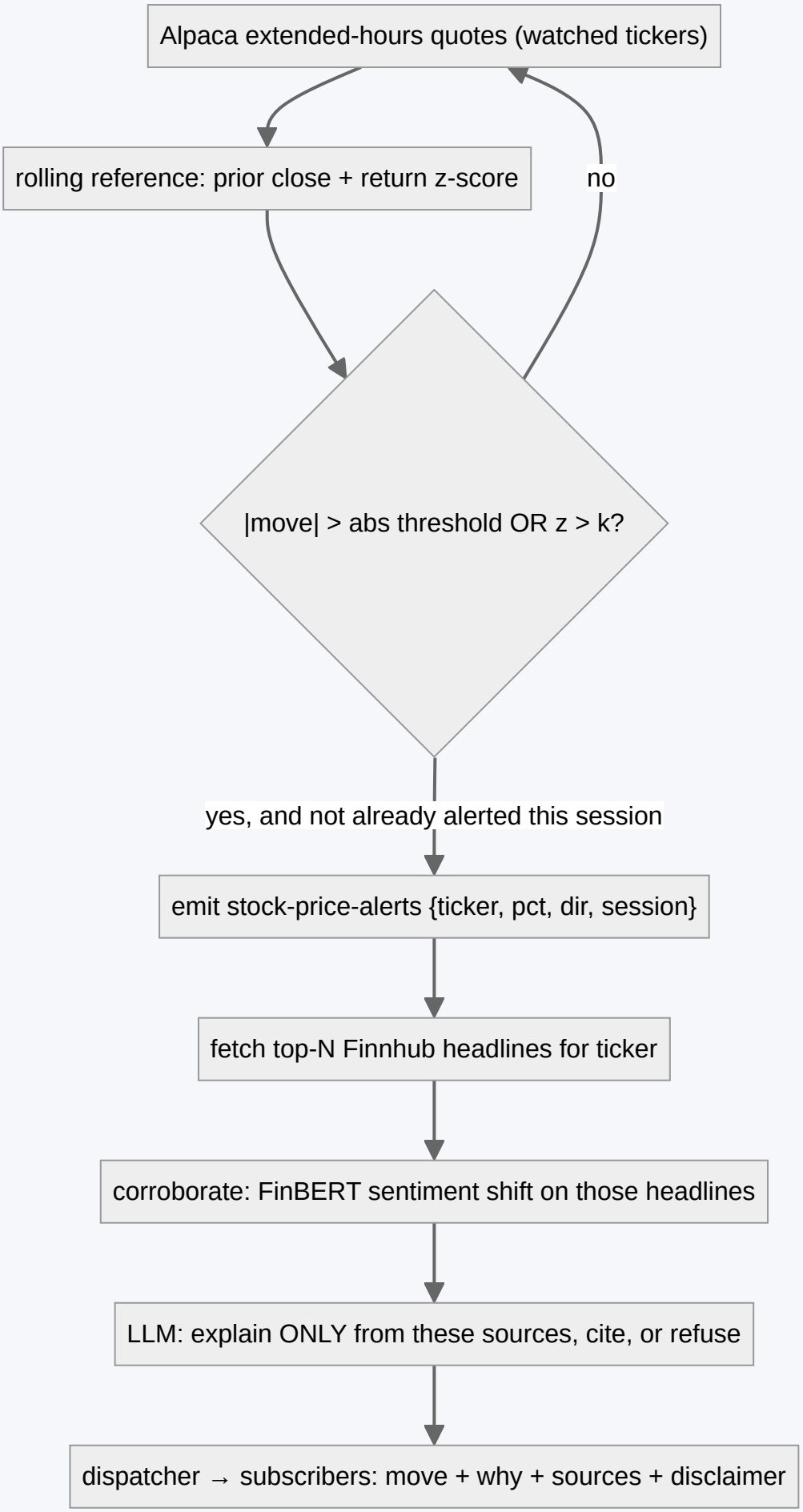
Alpaca extended-hours quotes (watched tickers)
rolling reference: prior close + return z-score
no
|move| > abs threshold OR z > k?
yes, and not already alerted this session
emit stock-price-alerts {ticker, pct, dir, session}
fetch top-N Finnhub headlines for ticker
corroborate: FinBERT sentiment shift on those headlines
LLM: explain ONLY from these sources, cite, or refuse
dispatcher → subscribers: move + why + sources + disclaimer

Fig 3 — Detection is deterministic (thresholds/z-score); explanation is retrieval-grounded. If retrieval is empty, the alert still sends the move with an honest "no clear catalyst in the news yet."

Detection is pure arithmetic — a percent move vs. prior close and a z-score of recent returns, both above configurable thresholds — so it never depends on a model being up. **Explanation** retrieves the actual headlines, optionally corroborates with the sentiment already scored on them, and asks the LLM to explain strictly from those sources with citations. Full execution spec in §17.

5.4 Macro release detection & explanation — to build

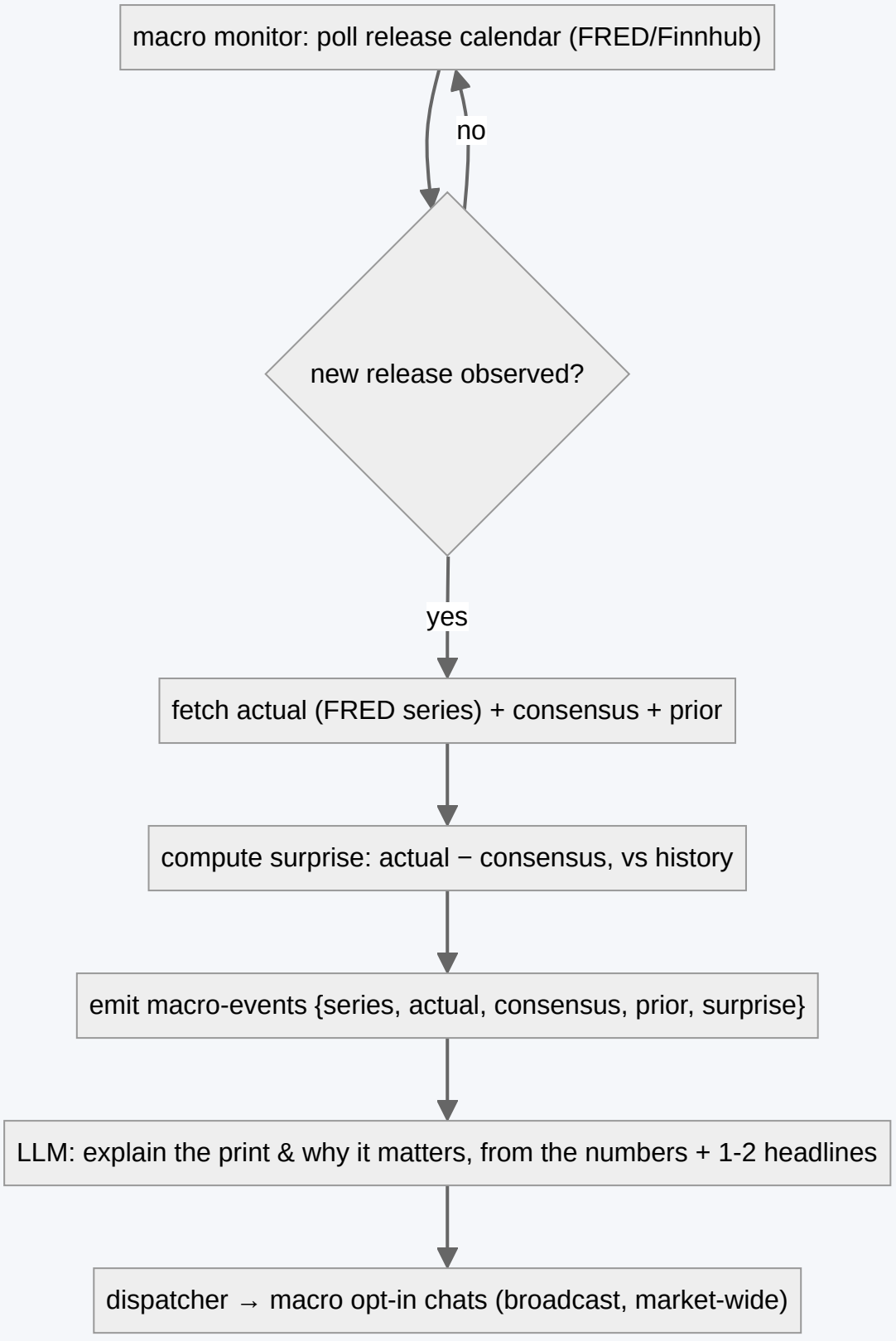

Fig 4 — Macro is market-wide and low-volume: a handful of scheduled prints a week, each broadcast once, each explained from the numbers plus a couple of grounding headlines.

5.5 On-demand Q&A (`/why`) — to build

A pull version of 5.3: the user asks “why is $TSLA moving?” and the same retrieval+explain path runs on demand for the active ticker's most recent notable move, cache-first so repeated asks are instant and identical.

5.6 Cross-cutting rules

- **One model per record type.** FinBERT on every post; the LLM only on events. If an event explainer is making more than one LLM call per event, it's wrong (retrieval + one grounded generation).
- **Everything grounded or refused.** No explainer emits a “why” it can't tie to a retrieved source.
- **Idempotent & de-duplicated.** Every pipeline keys on a stable id / natural key; a replay or re-poll produces no second alert.
- **Logged, never leaking.** Pipeline name, latency, model, token counts, outcome — not full post bodies or PII (§10).

06 NLP & the FinBERT Sentiment Engine

BRIEF §3 · THE ML PIPELINE

This is the “own ML pipeline” the brief asks for: a real, domain-specific NLP model wired into a streaming system — not an LLM prompt wearing a lab coat.

6.1 The model

`ml_engine/finbert_model.py` wraps `yiyanghkust/finbert-tone` (labels: neutral / positive / negative). `predict_batch` tokenises with padding+truncation at 128 tokens, runs a single forward pass under `torch.inference_mode()`, softmaxes the logits, and returns a `SentimentResult` whose `sentiment_score` property is **P(positive) − P(negative) ∈ [−1, 1]**. A process-local singleton (`get_shared_model`) ensures weights load once per worker, since loading — not inference — is the expensive part.

6.2 Distributed inference

Two paths, one model wrapper:

PATH	WHERE	USE
`score_dataframe_with_spark` (`mapInPandas`)	Inside the streaming `foreachBatch`	**Production.** One model per partition; batched; the scalable hot path.
`RayInferencePool` (actor pool)	Standalone/batch/notebook	Backfills and ad-hoc scoring; N actors, round-robin batches; each actor holds one loaded model.

Both funnel through `predict_batch`, so behaviour is identical and there is exactly one place where inference semantics live.

6.3 From scores to a leading indicator

Raw per-post scores are noisy. The pipeline that turns them into signal:

```
scored_sentiment            # per-post P(pos)-P(neg)
  → aggregate_sentiment(window="15min")     # mean score + post_count per (ticker, window), stamped at window CLOSE
  → merge_asof(price, direction="backward", allow_exact_matches=False)   # point-in-time join
  → add_rolling_sentiment({"15m","1h","4h","24h"})    # rolling means, still lookahead-safe
  → generate_signals(entry=+0.4, exit=-0.2)           # CROSSINGS above/below thresholds
```

Why crossings, not levels: a level signal (“sentiment is above 0.4”) stays true for many bars and produces one giant position; a crossing (“rose *through* 0.4 this bar”) is a discrete event the backtester can enter and exit on. `generate_signals` uses `sentiment > t & prev_sentiment <= t`.

6.4 Proving it leads — the honest part

The claim “sentiment is a leading indicator” is only worth making if it survives a backtest that cannot cheat. Two guards make the vectorbt result (`quant_backtest/backtester.py`) trustworthy:

1. **No lookahead:** `allow_exact_matches=False` means a price bar at `t` sees only sentiment aggregated strictly before `t`. Rolling features inherit this — a window ending at `t` uses only rows ≤ `t`.
2. **Benchmarked:** every strategy is run against `Portfolio.from_holding` (buy-and-hold). The bot reports **total return, Sharpe, max drawdown, and the delta vs. buy-and-hold** — if the “signal” doesn't beat holding, that shows.

> **THRESHOLD ENHANCEMENT (PLANNED)**
>
> The brief asks for an alert when sentiment moves “beyond a certain threshold”. The current fixed ±0.4/−0.2 crossing is a good v1. The stronger leading-indicator form is a **rolling z-score**: alert when a ticker's short-window sentiment is *k* standard deviations above its own recent baseline (and volume of posts is non-trivial). This adapts per ticker, catches *changes* rather than absolute levels, and is a small addition to `signal_generator.py` (milestone A4, §13).

6.5 Preprocessing & known NLP limits

- **Truncation at 128 tokens** — long DD posts lose their tail. Acceptable for headline-register news; a chunk-and-average pass is a future refinement for long-form.
- **Register mismatch** — FinBERT is trained on financial news/filings, so dense WSB slang, sarcasm, and emoji-as-sentiment are its weak spot. Named plainly in §16; the mitigation is aggregation (one sarcastic post barely moves a windowed mean) and, later, fine-tuning on labelled cashtag data.
- **Neutral inflation** — low-content posts skew neutral, which is correct behaviour, not a bug; the `post_count` field lets the signal layer ignore thin windows.
- **Language** — English only for v1. Non-English posts should be filtered at ingestion rather than mis-scored.

07 Feature-by-Feature Implementation

BRIEF §2 · THE THREE FEATURES

Each feature: what it is, what it depends on, the data it reads/writes, its error and edge cases, and the guardrail that keeps it honest.

7.1 Feature 1 — Social/news sentiment leading indicator MOSTLY BUILT

What it does

Continuously scores forum/news posts with FinBERT, aggregates per ticker/window, and pings a subscriber the moment rolling sentiment *crosses* the entry/exit threshold — designed to move before the clean price move does, and validated by a buy-and-hold-benchmarked backtest.

Depends on

Kafka + Spark + Delta + FinBERT (built). Real Reddit/X/RSS clients (to wire). Optional z-score threshold (to add).

ASPECT	DETAIL
Reads/writes	Reads `scored_sentiment` + OHLCV; writes nothing user-facing except the alert. Backtest computed on demand, TTL-cached.
Subscription	`/subscribe` sets `ChatState.subscribed`; the `JobQueue` loop checks the latest bar per subscriber.
De-dup	`last_alert[ticker]` = bar timestamp; a crossing is announced once.
Errors	Empty/late data → loop skips silently; one chat's exception is caught so the loop survives; unknown ticker rejected at `/ticker`.
Guardrail	The alert states the signal and the bar time; it never says “buy”. The `/backtest` stats are shown honestly, including underperformance.

7.2 Feature 2 — Pre/post-market price-move alerts & the “why” TO BUILD

User subscribes to a ticker (`/watch NVDA`); the price monitor watches extended-hours quotes; a significant gap fires an alert whose body answers *why*, grounded in the headlines that broke.

ASPECT	DETAIL
APIs	Alpaca real-time (extended-hours quotes) · Finnhub company-news (retrieval) · LLM (explanation) · Telegram (push).
New modules	`ingestion/price_monitor.py`, `explain/news_retrieval.py`, `explain/llm_provider.py`, `telegram_bot/dispatcher.py`.
Detection	% move vs. prior close AND/OR return z-score over recent bars; both thresholds in `settings`; session-aware (pre 04:00–09:30 ET, post 16:00–20:00 ET).
Explanation	Retrieve top-N recent headlines → LLM explains strictly from them, cites, or says “no clear catalyst yet” (§17).
Reads/writes	Writes `stock-price-alerts` Kafka + an optional `price_alerts` Delta audit table; reads news at explain time.
Subscription	`watched_tickers` per chat; dispatcher fans out only to watchers of that ticker.
De-dup	One alert per (ticker, session, direction) unless the move materially extends; a re-observed quote never re-fires.
Errors	News empty → send the move with an honest no-catalyst note. LLM down → send move + raw top headline, no synthesis. Feed gap → don't invent a move from a stale quote.
Guardrail	GUARD-1: explanation is retrieval-grounded and cited, never speculative; every alert carries the not-advice disclaimer (§16).

7.3 Feature 3 — US macroeconomic release alerts & the “why” TO BUILD

When the Fed / BLS / BEA releases a key number, every opted-in chat gets one message: what printed, vs. consensus and prior, and why it matters — explained from the numbers plus a wire headline or two.

ASPECT	DETAIL
APIs	FRED (series + release calendar) · Finnhub economic calendar (consensus) · BLS/BEA (context) · LLM · Telegram.
New modules	`ingestion/macro_monitor.py`, reuses `explain/` and `dispatcher.py`.
Detection	Poll the calendar; when a scheduled series posts a new observation, compute surprise = actual − consensus, scaled by the series' historical surprise dispersion.
Coverage (v1)	FOMC rate decision, CPI, PCE, Non-farm Payrolls, Unemployment, GDP. A small, curated, high-impact set.
Reads/writes	Writes `macro-events` Kafka; broadcast (market-wide), not per-ticker.
Subscription	`/macro on|off` → `macro_opt_in`.
De-dup	Key on (series, release_time); a revised figure is a distinct, clearly-labelled “revision” alert, not a duplicate.
Errors	Consensus missing → report actual vs. prior only. Number not yet posted at scheduled time → keep polling, don't announce a blank.
Guardrail	Explains, never forecasts Fed policy or tells anyone to trade the print. Numbers quoted are the retrieved figures, verbatim.

7.4 Shared subscription model

All three features share `ChatState` and one fan-out discipline: look up interested chats, respect a per-event de-dup key, and send through one code path so throttling and formatting are uniform. Feature 1 is *polled* off Delta (signal lives in a table); Features 2–3 are *pushed* off Kafka (events arrive as they happen). The user experiences all three as “the bot messaged me with something that matters, and told me why.”

08 Data Layer: Kafka Topics & Delta Tables

BRIEF §3 · STORAGE

8.1 Kafka topics

TOPIC	KEY	VALUE SCHEMA	PRODUCER → CONSUMER
`stock-raw-sentiment`	ticker	id, ticker, timestamp, source, text	social/news producer → Spark scorer BUILT
`stock-scored-sentiment`	ticker	+ positive, neutral, negative, sentiment_score	(reserved for a Kafka-native scored stream) RESERVED
`stock-price-alerts`	ticker	ticker, pct_move, direction, session, ref_price, ts	price monitor → dispatcher TO BUILD
`macro-events`	series	series, actual, consensus, prior, surprise, release_time	macro monitor → dispatcher TO BUILD

3 partitions, RF=1 in dev (`ensure_topics`). Production: RF≥3 and partition count sized to consumer parallelism.

8.2 Delta Lake tables

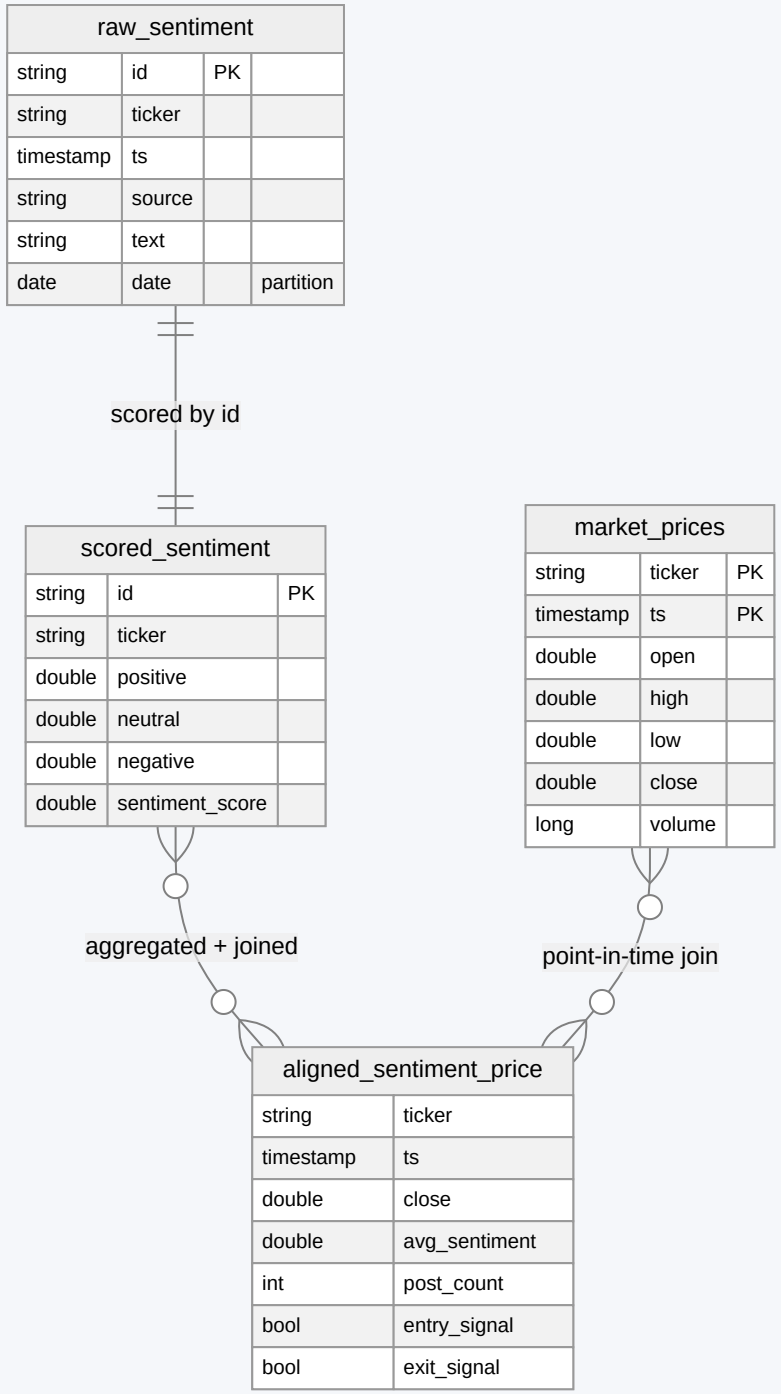

Fig 5 — The lakehouse. `raw` is the immutable audit trail; `scored` is the re-computable model output; `aligned` is the point-in-time research/serving view.

TABLE	KEY (MERGE)	PARTITION	PURPOSE
`raw_sentiment`	append-only	date, ticker	Verbatim audit trail; source of any re-score.
`scored_sentiment`	`id`	date, ticker	FinBERT output; idempotent upsert.
`market_prices`	`(ticker, timestamp)`	date, ticker	OHLCV bars; idempotent on backfill.
`aligned_sentiment_price`	derived	date, ticker	Point-in-time sentiment+price+signals for serving.
`price_alerts` NEW	`(ticker, session, ts)`	date	Feature 2 audit: what fired, what was said, which sources.
`macro_events` NEW	`(series, release_time)`	date	Feature 3 audit of every explained release.

8.3 Bot state (not in Delta)

Per-chat preferences live in a lock-guarded JSON file (`telegram_bot/state.py`), deliberately outside the lakehouse — they're tiny, mutable, and unrelated to the analytical tables. Fields today: `ticker` , `window` , `subscribed` , `last_alert{}` ; Feature 2/3 add `watched_tickers[]` ,

`macro_opt_in`, and a per-event `last_alert` namespace.

8.4 Maintenance

- **Compaction:** `optimize_table()` (+ optional ZORDER by ticker) counters the small-file churn from 30-second micro-batch appends. Schedule nightly.
- **Retention:** raw/scored kept for research; a VACUUM policy caps time-travel history. Alert audit tables kept for post-mortems.
- **Re-score:** swap the model, truncate `scored_sentiment`, replay from `raw_sentiment` — Kafka untouched.

09 Services, Endpoints & Background Jobs

BRIEF §3 · RUNTIME

Mostly long-running services and scheduled loops rather than a REST API — the interface is Telegram, so there's little to expose over HTTP beyond health and an optional webhook.

9.1 Services (Compose)

SERVICE	COMMAND	ROLE
`zookeeper` / `kafka`	Confluent images	The bus; healthchecks gate dependents. BUILT
`kafka-producer`	`python -m ingestion.kafka_producer --poll 60`	Real RSS/Reddit/Finnhub ingestion. BUILT
`ml-inference-spark`	`python -m data_pipeline.spark_streaming`	The scoring stream; CPU/mem reservation. BUILT
`telegram-bot`	`python -m telegram_bot.bot`	Commands + polled sentiment alerts. BUILT
`price-monitor`	`python -m ingestion.price_monitor`	Extended-hours detection → Kafka. TO BUILD
`macro-monitor`	`python -m ingestion.macro_monitor`	Release-calendar poll → Kafka. TO BUILD
`alert-dispatcher`	`python -m telegram_bot.dispatcher`	Consume alert topics → explain → fan out. TO BUILD
`dashboard` (opt)	`streamlit run dashboard/app.py`	Builder-facing charts. BUILT

9.2 Background jobs

JOB	CADENCE	PURPOSE
Spark micro-batch	every 30s (`trigger(processingTime)`)	Score & land the sentiment stream.
`check_and_send_alerts`	`JobQueue` , 60s	Poll latest bar per subscriber; push fresh crossings.
price-monitor loop	websocket-driven	Evaluate each quote against thresholds; emit alerts.
macro-monitor poll	every ~15 min (tighter near scheduled release times)	Detect new releases; emit macro events.
Delta OPTIMIZE	nightly	Compact small files; ZORDER by ticker.
news/quote cache TTL	per request (`_TTLCache`)	Collapse bursts; keep repeated `/why` instant & consistent.

9.3 Alert dispatch sequence (Features 2–3)

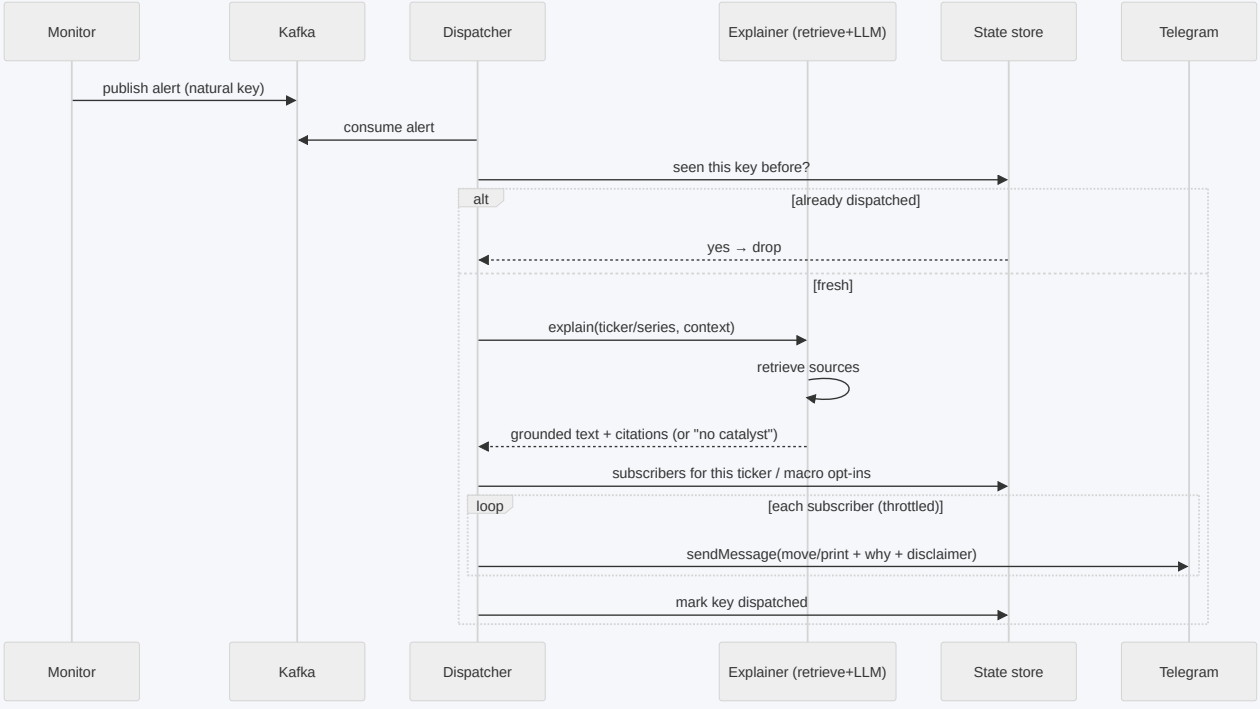

Fig 6 — Explain once per event, then fan out. De-dup is checked before the (costly) LLM call, so a re-consumed message never re-explains.

9.4 HTTP surface (minimal)

ENDPOINT	AUTH	PURPOSE
`GET /health`	none	Liveness for each service.
`GET /health/deps`	admin token	Probe Kafka, Delta path, FinBERT load, LLM, data APIs — run before a demo.
`POST /telegram/webhook`	Telegram secret token	Scale-path alternative to polling (§03, §14).
`POST /internal/simulate/alert`	admin token	Inject a fake price/macro event to demo the dispatch path without waiting for the market.

10 Security, Logging & Error Handling

BRIEF §3 · OPS

10.1 Security

- **Secrets via env only.** `TELEGRAM_BOT_TOKEN`, `ALPACA_*`, FRED/Finnhub keys, the LLM key — all through `config/settings.py` from the environment; `.env` is git-ignored, `.env.example` is committed with empty values. Compose fails loudly if the bot token is missing.
- **Least privilege on market APIs.** Alpaca **paper** keys and read-only data scopes; the system has no code path that can place an order. That's a deliberate property, not an omission — it can never trade, by construction.
- **Telegram trust.** Polling authenticates via the bot token; a webhook deployment adds Telegram's secret-token header check. Callback-query data is validated against known prefixes, never `eval`'d.
- **Input hygiene.** Every inbound/source string is `html.escape()`-d before it enters an HTML message; ticker/window inputs are checked against allow-lists; message-length caps before anything hits the LLM.
- **Prompt-injection defence.** Retrieved headlines are untrusted text. They go into the LLM inside a clearly delimited block, the system prompt states that block is data (not instructions), and — the real backstop — the output is only ever a short explanation shown to the user, never a tool call or a trade. A successful injection can at worst produce a wrong sentence, which the citation requirement and the disclaimer already bound. See §16.
- **PII.** There is essentially none — chat IDs and preferences only. Chat IDs are never sent to the LLM or any data API.
- DEFERRED **pydantic-settings** typed config validation, secret rotation, per-chat auth for any future admin commands.

10.2 Logging

LEVEL	CONTENT
INFO	Batch sizes & counts (Spark), messages produced, alert sent (chat_id, ticker, bar), LLM call (pipeline, tokens, latency), job start/finish.
WARNING	Kafka topic-create failure, model reload, empty/late data skip, LLM rate-limit backoff, news retrieval empty, provider fallback.
ERROR	Send failure after retries, unhandled pipeline exception, Delta write error, feed disconnect.
CRITICAL	Scoring stream down, dispatcher unable to reach Telegram, a data API returning a price move that fails sanity bounds.

Never logged: full post bodies, LLM prompt/response text verbatim, API keys, the bot token. Log the *shape* (id, ticker, length, outcome), not the content. A correlation id per event threads a price/macro alert through detect → explain → dispatch.

10.3 Error handling — universal rules

1. **Every external call has a timeout** (Kafka, yfinance/Alpaca, news, FRED, LLM, Telegram). No exceptions.
2. **Every alert has a fallback.** No news → send the move with an honest "no clear catalyst yet." No LLM → send the move + the top raw headline. Never a stack trace, never silence on a real event.
3. **The stream fails safe.** Raw is written before scoring; a scoring failure loses no data (re-score later).
4. **One subscriber's failure never stops the loop** (already true in `alerts.py`).

FAILURE	HANDLING	USER SEES
Kafka unreachable	Producer retries (`acks=all`, retries=5); Spark waits, Kafka retains.	Slight lag, no loss.
FinBERT load fails	Raw table still fills; alert on CRITICAL; backfill re-scores.	Sentiment charts stale until recovered.
yfinance rate-limit / gap	Backoff; serve cached bars; Alpaca fallback if configured.	Cached chart; a refresh retries.
News API empty/429	Skip synthesis; send move with no-catalyst note; back off.	"$NVDA −6% pre-market. No clear catalyst in the news yet."
LLM 429 / timeout	Backoff + jitter; degrade to raw-headline mode; cache prior explanation.	Move + one real headline, no synthesis.
Telegram 429 (flood)	Respect `retry_after`; throttle fan-out to ≤~1 msg/s per chat.	Alerts arrive a beat later, in order.
Stale/again quote	Sanity-bound the move; ignore a quote older than N seconds; never fabricate a gap.	Nothing — no false alert.
Duplicate event (replay/re-poll)	Natural-key de-dup before explain/send.	Nothing — no double alert.

11 Testing Strategy

BRIEF §3 · QUALITY

Scoped for delivery: enough tests to stop the demo breaking and to protect the two things that must never be wrong — the point-in-time guarantee and the “not advice / grounded explanation” guardrails.

11.1 Priorities (stop at the line if time is short)

1. **Point-in-time alignment** — a bar must never see sentiment at or after its own timestamp. The whole backtest depends on it.
2. **Signal crossing logic** — entry/exit fire on a transition, not a level; edge cases at the threshold.
3. **Explainer grounding** — no sources ⇒ refusal / no-catalyst; every claim ties to a retrieved headline.
4. **Alert de-dup** — one crossing / one event ⇒ exactly one message.
5. **FinBERT scoring contract** — score ∈ [−1,1]; positive/negative text lands on the right sign.
6. **Bot handlers** — unknown ticker/window rejected; HTML escaping; state transitions. — cut line —
7. Delta upsert idempotency; backtest stats sanity; degraded-mode paths.

11.2 Unit tests

TARGET	CASES
`align_sentiment_with_price`	sentiment strictly before bar (used) · exactly at bar (excluded) · after bar (excluded) · no sentiment (zeros, no crash) · window right-edge stamping.
`generate_signals`	cross above entry (fires once) · stays above (no re-fire) · cross below exit · hover at threshold · first bar (NaN prev → False).
`aggregate_sentiment`	bucket-close timestamp · empty windows dropped · post_count > 0 filter.
FinBERT wrapper	batch = single consistency · score bounds · empty list · truncation at 128.
explainer	empty retrieval → refusal string · injection attempt in a headline → ignored, no leaked instruction · citations present.
de-dup	same bar/event twice → one send · revision → distinct labelled send.
bot handlers	bad ticker, bad window, escaping, `/status` reflects state.

11.3 Integration tests

- Produce N messages → Spark micro-batch (local `SparkSession`) → assert rows in raw + scored Delta, no dupes on replay.
- Scored + price fixtures → align → signals → backtest returns finite stats and a benchmark.
- Inject a fake `stock-price-alerts` record → dispatcher → mocked news + mocked LLM → assert one grounded message to the right subscribers.
- Inject a fake `macro-events` record → broadcast to opt-ins only; a second identical record sends nothing.

11.4 Mocking external services

SERVICE	APPROACH
Kafka	Ephemeral broker in CI (testcontainers) or an in-memory fake for pure-logic tests.
yfinance / Alpaca	Recorded OHLCV fixtures (include an extended-hours gap and a thin-volume ticker).
News / FRED	Fixture JSON captured once; include an empty-news case and a missing-consensus case.
LLM	Stub provider returning canned grounded/refusal outputs; one real call only in the smoke test. Also test a 429 and a malformed response.
Telegram	Mock `bot.send_message` / `send_photo`; assert call count, target chats, and that media sends carry no stray markup.

11.5 Pre-demo smoke test

- `docker compose up` → producer + Spark fill Delta; `/price` and `/backtest` return charts within a minute.
- `/health/deps` green for Kafka, Delta path, FinBERT, LLM, Alpaca, Finnhub, FRED.
- Inject a fake price alert and a fake macro event via `/internal/simulate/alert`; both arrive explained, once, with disclaimer.
- Subscribe, force a crossing in seed data, confirm exactly one push naming the bar time.
- LLM key quota not exhausted by rehearsal; Alpaca/Finnhub/FRED keys valid.

12 Deployment & Folder Structure

BRIEF §3 · DEPLOY

12.1 Compose topology

One `docker-compose.yml`, one image (the `Dockerfile` installs OpenJDK 17 for Spark plus build tools for torch), each service overriding the command. Healthchecks gate order: ZK → Kafka → {producer, Spark, monitors} → bot/dispatcher. The Spark service carries a CPU/memory reservation (FinBERT is the hot path). A shared `./data` volume holds the Delta lake, checkpoints, and the bot's JSON state.

```
# added for Features 2-3, same image, new commands
  price-monitor:    command: ["python","-m","ingestion.price_monitor"]
  macro-monitor:    command: ["python","-m","ingestion.macro_monitor"]
  alert-dispatcher: command: ["python","-m","telegram_bot.dispatcher"]
```

12.2 Environment

```
TELEGRAM_BOT_TOKEN=            # @BotFather
KAFKA_BOOTSTRAP_SERVERS=kafka:29092
TICKERS=AAPL,NVDA,TSLA,MSFT
FINBERT_MODEL_NAME=yiyanghkust/finbert-tone
MARKET_DATA_PROVIDER=yfinance   # or alpaca (live extended hours)
ALPACA_API_KEY= / ALPACA_API_SECRET=
FINNHUB_API_KEY=  FRED_API_KEY=   # Features 2-3
LLM_PROVIDER=claude  LLM_API_KEY=  # explainer, behind LLMProvider
ENTRY_SENTIMENT_THRESHOLD=0.4  EXIT_SENTIMENT_THRESHOLD=-0.2
PRICE_MOVE_PCT=3.0  PRICE_MOVE_Z=2.5   # Feature 2 detection
```

Every value already resolves through `config/settings.py` with an env override, so the same code runs on a laptop (localhost) or in compose (service hostnames).

12.3 Folder structure

```
stock-sentiment-analysis-pipeline/
├── config/settings.py            # one settings module, env-overridable
├── ingestion/
│   ├── kafka_producer.py         # real sources → Kafka (poll loop, de-dup)
│   ├── kafka_utils.py            # topics, (de)serialize, producer/consumer
│   ├── sources/                  # Source interface + RSS/Reddit/Finnhub + ticker extract
│   ├── price_monitor.py          # [new] extended-hours detection
│   └── macro_monitor.py          # [new] release-calendar poll
├── data_pipeline/
│   ├── spark_streaming.py        # Kafka → FinBERT → Delta (foreachBatch)
│   ├── delta_writer.py           # MERGE upserts, partition, OPTIMIZE
│   └── data_alignment.py         # OHLCV fetch + point-in-time merge_asof
├── ml_engine/
│   ├── finbert_model.py          # model wrapper + singleton
│   └── distributed_inference.py  # Ray pool + Spark pandas-UDF
├── quant_backtest/
│   ├── signal_generator.py       # rolling sentiment + crossings (+ z-score)
│   └── backtester.py             # vectorbt vs buy-and-hold
├── explain/                      # [new] grounded event explainer
│   ├── llm_provider.py           # LLMProvider (Claude default, pluggable)
│   ├── news_retrieval.py         # Finnhub/RSS per-ticker headlines
│   └── prompts.py                # grounded, cited, refuse-if-thin
├── telegram_bot/
│   ├── bot.py handlers.py keyboards.py state.py
│   ├── alerts.py                 # polled sentiment crossings
│   ├── dispatcher.py             # [new] Kafka alert fan-out + explain
│   ├── charts.py data_service.py
├── dashboard/app.py              # Streamlit (builder-facing)
├── docs/                         # this blueprint
├── tests/{unit,integration,fixtures}
├── docker-compose.yml  Dockerfile  requirements.txt  .env.example
```

Structural rule: `data_service.py` and the future `dispatcher.py` are the two serving choke points; only `ml_engine/` imports transformers/torch, and only `explain/` imports the LLM SDK. If any other module reaches for those, something has gone wrong.

13 Development Roadmap

BRIEF §3 · MILESTONES PER FEATURE

One shared foundation, then **three independent feature tracks** — each a sequence of milestones with a concrete, testable **"Done when"**. Hand one milestone at a time to an implementer; do not hand over a whole track and ask for "the feature".

Legend: BUILT code complete (logic unit-tested) PARTIAL exists but needs work TO BUILD new.

BUILD LOG — 2026-08-28

Build pass 1: the synthetic post generator was removed and replaced with real ingestion sources (**A5**); the rolling z-score anomaly signal was added (**A4**). Both are code-complete with green unit tests. Note on verification: **live** end-to-end runs (Kafka + Spark + the FinBERT model download) and every keyed source execute in **your Docker environment** with your credentials — the build sandbox has no network path to Telegram, Reddit, Alpaca, Finnhub, FRED, or HuggingFace, so a "built" pill here means code-complete and logic-tested, with live verification handed to you. Credentials needed per track are listed at the end of this section.

Track 0 — Platform foundation

SHARED BACKBONE

M0.1 Streaming spine — BUILT

Kafka + ZK in Compose; `kafka_producer` emits ticker-tagged messages; `ensure_topics` idempotent.

Done when: `docker compose up` shows the producer publishing to `stock-raw-sentiment` and `kafka-broker-api-versions` healthcheck is green.

M0.2 Scoring stream → lakehouse — BUILT

Spark `foreachBatch`: parse → dedupe → raw Delta → FinBERT (`mapInPandas`) → scored Delta MERGE.

Done when: after a minute of streaming, `scored_sentiment` has rows with `sentiment_score ∈ [−1,1]` and replaying a partition adds zero duplicates.

M0.3 Bot skeleton + data service — BUILT

`python -m telegram_bot.bot` polling; TTL-cached `data_service`; `/start /status /refresh`.

Done when: the bot replies to `/start` and `/status` reflects a changed ticker.

Track A — Feature 1: Sentiment leading indicator

SOCIAL / NEWS → ALERT

A1 Point-in-time alignment + signals — BUILT

`merge_asof` backward/exact=False; rolling sentiment; entry/exit crossings.

Done when: a unit test proves a bar at `t` never sees sentiment ≥ `t`, and a crossing fires exactly once per transition.

A2 Backtest & charts in-chat — BUILT

vectorbt strategy vs. buy-and-hold; `/price`, `/backtest`, `/posts` as PNGs + stats.

Done when: `/backtest NVDA` returns a chart and total-return / Sharpe / drawdown vs. benchmark.

A3 Subscription alerts — BUILT

`/subscribe`; `JobQueue` loop pushes fresh crossings; de-dup on bar timestamp.

Done when: forcing a crossing in seed data delivers exactly one push naming the bar time, and never a second for the same bar.

A4 Rolling z-score threshold ("beyond a threshold") — BUILT

Per-ticker rolling z-score in `signal_generator.py` (`add_sentiment_zscore` / `generate_zscore_signals`): fires when sentiment is *k* σ from its trailing baseline — computed against a baseline shifted one bar back, so the current value never inflates its own distribution (lookahead-safe) — with at least `SENTIMENT_Z_MIN_POSTS` posts in the window.

Done when: a ticker whose sentiment jumps against its own baseline triggers, while a ticker at a high-but-flat level does not; both covered by unit tests. ✓ `tests/test_zscore_signals.py` (15 tests green).

A5 **Real ingestion sources** BUILT

Synthetic generator removed. `ingestion/sources/` ships three real sources behind one `Source` interface: keyless **RSS** (default, no credentials), **Reddit** (PRAW), and **Finnhub** news; cashtag + allow-list ticker extraction (`ticker_extract.py`); the producer poll-loop de-dups and keeps the message contract.

Done when: real posts about a universe ticker appear scored in `scored_sentiment` within one micro-batch, correctly tagged. ✓ Code + transform/extract unit tests green (`tests/test_rss_source.py` , `tests/test_ticker_extract.py`). **Live run needs your env:** RSS works with zero keys; Reddit/Finnhub activate when their keys are set (§ credentials).

A6 **Signal validation report** TO BUILD

A scheduled out-of-sample backtest summary per ticker so “leading indicator” stays an evidenced claim, not a slogan.

Done when: a weekly job writes per-ticker strategy-vs-benchmark stats and the bot can surface the latest on request.

Track B — Feature 2: Pre/post-market price-move + why

TAPE → EXPLANATION

B1 **Watch subscriptions** TO BUILD

`/watch SYM` / `/unwatch SYM` ; `watched_tickers` on `ChatState` .

Done when: `/watch NVDA` persists across restart and `/status` lists watched tickers.

B2 **Extended-hours price monitor** TO BUILD

`ingestion/price_monitor.py` : Alpaca websocket for watched tickers; % move vs. prior close + return z-score; session-aware; emit `stock-price-alerts` .

Done when: a simulated pre-market −5% gap on a watched ticker produces exactly one `stock-price-alerts` record with the correct pct/direction/session, and a 0.3% wiggle produces none.

B3 **News retrieval + LLM explainer** TO BUILD

`explain/news_retrieval.py` (Finnhub top-N) + `explain/llm_provider.py` + grounded, cited prompt; refuse when thin. Full spec §17.

Done when: given a real ticker + headlines the explainer returns a <60-word cited “why”; given empty headlines it returns the no-catalyst line, never a fabricated reason (unit-tested).

B4 **Dispatcher fan-out** TO BUILD

`telegram_bot/dispatcher.py` : consume `stock-price-alerts` , de-dup on key, explain once, fan out to watchers with disclaimer; throttle to Telegram limits.

Done when: one injected alert reaches only the ticker's watchers, once, explained; a replay of the same record sends nothing.

B5 **On-demand `/why`** TO BUILD

Pull path reusing B3 for the active ticker's latest notable move; cache-first.

Done when: `/why` answers for a moving ticker and repeated asks return the identical cached explanation instantly.

B6 **Audit + degraded modes** TO BUILD

`price_alerts` Delta audit; graceful paths for empty news / LLM down / stale quote.

Done when: with the LLM stubbed to fail, alerts still send (move + raw headline) and every fired alert is recorded in `price_alerts` .

Track C — Feature 3: US macro release + why

WASHINGTON → BROADCAST

C1 **Macro opt-in** TO BUILD

`/macro on|off` → `macro_opt_in`.

Done when: opt-in persists and `/status` shows macro state.

C2 **Release calendar + detection** TO BUILD

`ingestion/macro_monitor.py`: poll FRED/Finnhub calendar for the curated set (FOMC, CPI, PCE, NFP, UNRATE, GDP); detect a new observation.

Done when: a mocked "CPI just posted" calendar entry is detected once and only once (natural key on series+release_time).

C3 **Actual-vs-consensus event** TO BUILD

Fetch actual (FRED) + consensus + prior; compute surprise scaled by historical dispersion; emit `macro-events`.

Done when: a released series yields a `macro-events` record carrying actual/consensus/prior/surprise; a missing consensus degrades to actual-vs-prior without error.

C4 **Explain + broadcast** TO BUILD

Reuse the explainer on the numbers + 1–2 wire headlines; dispatcher broadcasts to opt-ins (market-wide), de-dup on key; revisions labelled distinctly.

Done when: an injected macro event reaches every opt-in chat once, explains the print vs. consensus, and never advises a trade or forecasts policy (guardrail test).

C5 **Macro audit** TO BUILD

`macro_events` Delta audit of every explained release.

Done when: each broadcast has a corresponding audit row with the exact figures and text sent.

▲ DEMO-READY CUT LINE — Track 0 + A (1–5) + B (1–4) + C (1–4) is a complete three-feature demo ▲

Track H — Hardening & rehearsal

ABOVE THE LINE = UPSIDE

H1 **Guardrails & disclaimers everywhere** TO BUILD

Not-advice footer on every alert; grounding tests green; injection tests green (§16).

Done when: no code path can emit a trade recommendation or an un-cited "why", proven by tests.

H2 **Observability & health** TO BUILD

`/health/deps`, correlation ids, redacting logger, Delta OPTIMIZE nightly.

Done when: `/health/deps` is green across all deps and one event is traceable end-to-end by its correlation id.

H3 **Demo rehearsal** TO BUILD

Run the §11.5 smoke test twice on the deployed stack; inject one of each alert type.

Done when: the full script runs end-to-end twice without touching code.

Sequencing notes

- **Track A first** — it is the backbone Features 2–3 reuse (Kafka, Delta, the dispatcher pattern, the bot).
- **B before C** — B builds the `explain/` + `dispatcher.py` that C reuses wholesale; C is then mostly a new monitor.
- **B2 is the risk** — live extended-hours data quality and session handling. Spike it early with a replay of a known gap day rather than waiting for a live pre-market.
- **Grounding tests are not optional polish** — they are what make Features 2–3 safe to ship, so H1's tests are written alongside B3/C4, not after.

Manual prerequisites — credentials you must procure

Each is free to obtain. Put values in `.env` (gitignored; copy from `.env.example`) or set them as environment secrets — never in chat or in git. The build pauses for these where a milestone can't run without them.

UNBLOCKS	CREDENTIAL	WHERE TO GET IT	ENV VAR(S)
Run the bot at all	Telegram bot token	@BotFather → `/newbot`	`TELEGRAM_BOT_TOKEN`
A5 Reddit source (optional)	Reddit script app	reddit.com/prefs/apps → create “script” app	`REDDIT_CLIENT_ID/SECRET/USER_AGENT`
A5 news + B3 explainer retrieval	Finnhub API key	finnhub.io (free tier)	`FINNHUB_API_KEY`
B2 extended-hours quotes	Alpaca keys (paper)	alpaca.markets → paper account	`ALPACA_API_KEY/SECRET`
C2/C3 macro data	FRED API key	fredaccount.stlouisfed.org/apikeys	`FRED_API_KEY`
B3/C4 explanations	Anthropic API key	console.anthropic.com	`ANTHROPIC_API_KEY`

RSS ingestion (Feature 1 baseline) and yfinance research bars need **no** credentials, so the sentiment backbone runs on real data with only the Telegram token.

14 Cost, Scalability, Risks & Future Work

BRIEF §3 · THE HONEST SECTION

VERIFY BEFORE RELYING

Third-party free tiers and pricing move constantly. Re-check X API, Alpaca, Finnhub, FRED limits and the LLM's per-token rate against live pages before launch. The numbers below were reasonable at time of writing.

14.1 Cost

COMPONENT	STATUS	NOTES
Kafka, Spark, Delta, Ray, FinBERT, vectorbt, Streamlit, PTB	FREE	All OSS; the cost is compute to run them.
Compute (one host)	PAID	The real bill: a box with enough RAM/CPU for a JVM + Spark + torch. A single mid-tier VM or a small cloud instance. FinBERT on CPU is the driver.
yfinance / FRED / BLS	FREE	Unofficial (yfinance) or official free (FRED/BLS).
Alpaca / Finnhub	FREE TIER	Free tiers cover a demo and a small watchlist; paid tiers for full SIP / higher news rates.
X / Twitter API	PAID	The one genuinely paid data source. Reddit-only is a viable, free v1.
LLM explainer	FREE TIER → PAID	Low volume (events, not posts) keeps this small; a Haiku-class tier + caching is cents-scale for a demo.

Where it breaks: compute first (Spark + torch are memory-hungry — the demo host, not the APIs, is the early ceiling), then X API cost if social breadth grows, then LLM spend only if the explainer is (wrongly) put on a high-volume path instead of events.

14.2 Scalability — the order it breaks, and the fix

1. **Single Kafka broker / RF=1** (any real load) → multi-broker, RF≥3, partitions sized to consumers.
2. **FinBERT CPU throughput** (high post volume) → GPU executors, an ONNX-quantised head, or a dedicated inference service (Triton). The `mapInPandas` shape already parallelises across executors.
3. **Bot polling + JSON state** (many chats / multi-instance) → Telegram webhook + FastAPI; move state to Postgres/Redis; the dispatcher already decouples fan-out from the bot process.
4. **Pandas alignment in memory** (large universe) → push the aggregate+join into Spark/Delta instead of `toPandas()`.
5. **One host** (everything) → split into services on K8s; Kafka + Delta already make components independently scalable.

Caching that matters most: the `/why` and macro explanations keyed on (event, sources) — the same question is asked repeatedly around a move, so a short TTL cache has a high hit rate and keeps LLM spend and latency down. Already the pattern in `data_service._TTLCache`.

14.3 Risks & limitations

RISK	SEVERITY	MITIGATION
Financial harm from a bad signal/explanation	Highest	Structural not-advice stance (§16); grounded/cited explanations; backtest shown honestly; disclaimer on every alert.
Sentiment ≠ causation; markets are noisy	High	Point-in-time backtest with a benchmark; report underperformance; frame sentiment as one input, not a crystal ball.
FinBERT weak on slang/sarcasm/emoji	Medium	Aggregation dampens outliers; fine-tune on labelled cashtag data (future); state the limit plainly.
LLM hallucinated “why”	High	Retrieval-grounded + cited + refuse-if-thin; degrade to raw headline; never free-form.
Extended-hours data is thin/noisy	Medium	Session-aware thresholds; sanity-bound moves; z-score over recent bars; ignore stale quotes.
X API cost / access changes	Medium	Reddit-first design; X behind the same producer interface, added when budget allows.
Single broker / single host	Medium	Documented scale path; RF and replicas are config, not rewrites.
Look-ahead creeping back in	High if unnoticed	The point-in-time test is a required, permanent guard (§11).

14.4 Future work (value-to-effort order)

1. **Fine-tune the sentiment model** on labelled Reddit/X cashtag data — the biggest quality lever for Feature 1.
2. **Webhook + Postgres/Redis** for the bot — the prerequisite for scale and multi-instance.
3. **Volume & unusual-options signals** alongside sentiment — cheap corroboration for price-move alerts.
4. **Per-user watchlists & thresholds** — let users set their own move/z thresholds per ticker.
5. **RAG over filings/earnings transcripts** for richer explanations — the one place a vector DB earns its keep.
6. **Broaden the universe** and push alignment fully into Spark.
7. **Sentiment/return attribution dashboards** for the builder — which sources actually lead.

8. **Backtest-as-a-service** — let users backtest a sentiment threshold of their choosing from chat.

16 Bot Persona, Limitations & Guardrails

WHAT IT IS · CAN'T DO · WON'T DO

THE LOAD-BEARING DISCLAIMER

Sentinel is an **information tool, not financial advice**. It reports data, scores sentiment, and explains market and macro events from public sources. It does not recommend trades, predict prices, or manage money. Nothing it emits is a solicitation to buy or sell any security. This stance is enforced by what the code can output, not merely by this paragraph.

1 Persona

A fast, precise, plain-spoken market analyst that lives in a chat. It leads with the fact (the move, the score, the print), attaches the “why” when — and only when — it can ground it, names its sources, and stops. It never hypes (“to the moon”), never hedges into uselessness, and never pretends to certainty it doesn't have. Tone: a good desk analyst's morning note, not a finfluencer's thread.

Voice rules

- Fact first, reason second, source third. Every alert carries its timestamp/session.
- Numbers are the retrieved numbers, verbatim — never “about” or rounded away from the source.
- “Why” is cited or absent. “No clear catalyst in the news yet” is a valid, frequent, honest answer.
- No imperatives about trading. Describe the event; let the user decide.
- Sentiment and backtest results are shown with their uncertainty (benchmark delta, drawdown), not cherry-picked.

2 Limitations (named, not hidden)

AREA	LIMITATION
Model	FinBERT is tuned on financial news register; heavy slang, sarcasm, and emoji-sentiment degrade it. 128-token truncation clips long posts.
Causation	Sentiment leading price is a statistical tendency backtested per ticker, not a law. The bot shows the backtest, including when it fails.
Data	yfinance is unofficial and gappy; extended-hours quotes are thin; free-tier news/macro rates are limited; X is paid.
Coverage	Small ticker universe and a curated macro set by design; English only.
Explanation	Only as good as the retrieved sources; a real move with no published news yet gets an honest “no catalyst”, not a guess.
Scale	Single broker, single host, polling bot, JSON state — a demo/small-cohort shape, with a documented upgrade path (§14).
Latency	30-second micro-batch + 60-second alert poll: minutes-scale, not HFT. Appropriate for a leading-indicator/awareness tool.

3 Guardrails (structural)

These are enforced by architecture, so a prompt injection or a model error cannot bypass them.

GUARD-1 · EXPLANATIONS ARE GROUNDED OR REFUSED

The explainer only ever receives retrieved sources (headlines / release numbers) in a delimited block and is instructed to explain strictly from them, cite them, and return a no-catalyst refusal when they're thin. Its output is a short message shown to the user — never a tool call, never an order. The worst case of a successful injection is one wrong sentence, bounded by the citation requirement and the disclaimer.

GUARD-2 · NO TRADE RECOMMENDATIONS, EVER

There is no code path that emits “buy” / “sell” / a price target / position sizing. Alerts state what happened and why; the not-advice disclaimer rides on every one. Enforced by tests (H1).

GUARD-3 · NO MARKET ACCESS

Market keys are read-only / paper. The system cannot place, modify, or cancel an order. A compromised prompt or bug still cannot move money.

GUARD-4 · NO LOOK-AHEAD

Point-in-time alignment (`allow_exact_matches=False`) is a permanent, unit-tested invariant. A backtest that could see the future would manufacture false confidence — the one dishonesty this domain punishes hardest.

GUARD-5 · NUMBERS ARE VERBATIM & DE-DUPLICATED

Quoted figures are the retrieved values, unaltered; every alert is keyed so a replay/re-poll cannot double-send, and a revision is labelled a revision, not a duplicate.

4 How the three relate

The **persona** governs tone and may be imperfect and still safe. The **guardrails** govern outputs that could cause harm — grounding, no-advice, no-market-access, no-lookahead — and are structural so no amount of persona drift or injection bypasses them. The **limitations** are the honest boundary between the two: the places the system does not compensate, and says so, instead of quietly degrading.

17 Deep Spec — Price-Move Explanation Engine

FEATURE 2 EXECUTION SPEC

The hardest, most novel part to get right: turning “$NVDA −6% pre-market” into a sentence a trader trusts — grounded, cited, and honest when there's nothing to say. This section is written to be handed straight to an implementer (milestones B2–B4).

17.1 Detection contract (`ingestion/price_monitor.py`)

```
# Per watched ticker, on each extended-hours quote:
ref_price   = prior regular-session close                # the anchor a trader compares to
pct_move    = (last - ref_price) / ref_price * 100
ret_z       = zscore(recent_returns, lookback=N)         # adaptive, catches unusual moves on quiet names
session     = "pre" | "post" | "regular"                 # ET-clock derived

fire if  (abs(pct_move) >= PRICE_MOVE_PCT  OR  abs(ret_z) >= PRICE_MOVE_Z)
     and  session in {"pre","post"}
     and  quote is fresh (age < STALE_SECONDS)
     and  not already fired for (ticker, session, sign(pct_move))    # unless move materially extends

emit → stock-price-alerts: {ticker, pct_move, direction, session, ref_price, last, ts}
```

Detection is deterministic and model-free on purpose — it must fire whether or not the news API or LLM is up.

17.2 Retrieval (`explain/news_retrieval.py`)

- Fetch the top-N (default 5) most recent Finnhub company-news items for the ticker within a lookback window (default 24h, tightened toward the move time).
- Each source = {title, source_name, url, published_ts}. Drop items older than the move, dedupe by title.
- Optional corroboration: pull the FinBERT sentiment already scored on those same headlines from `scored_sentiment` — a sharp sentiment shift that lines up with the price move strengthens the explanation and is worth surfacing (“news flow turned sharply negative”).
- If retrieval is empty → skip the LLM entirely; the alert body is the no-catalyst line.

17.3 Grounded generation (`explain/prompts.py` , `llm_provider.py`)

```
# System (fixed):
"You explain why a stock moved, using ONLY the headlines provided.
 Cite each claim as [n]. If the headlines do not explain the move, say
 'No clear catalyst in the news yet.' Never give trading advice. <=60 words."

# User (per event):
"$NVDA moved {pct}% in {session}. Headlines:
 [1] {title} — {source} ({ts})
 [2] ...
 Explain the move from these only."
```

- The headlines block is untrusted data (GUARD-1): the model is told it is data, and its only possible output is text shown to the user.
- One LLM call per event, cache-keyed on (ticker, session, source-set hash) so a re-ask or a fan-out to many chats explains once.
- Validate the output: must be ≤ the word cap, must contain at least one `[n]` citation when headlines were supplied, must not contain imperative trade language (a small denylist check). On failure → degrade to “{move}. Top headline: [1] {title}.”

17.4 Dispatch (`telegram_bot/dispatcher.py`)

```
consume(stock-price-alerts)
  → key = (ticker, session, bucket(ts))
  → if seen(key): drop
  → explanation = explain(ticker, pct, session, sources)   # cached
  → for chat in watchers(ticker):  throttle; send(move + explanation + sources + disclaimer)
  → mark seen(key); write price_alerts audit row
```

Message shape sent to the user:

```
🔻 $NVDA  −6.1%  pre-market  (ref 118.20 → 110.98, 07:42 ET)
Why: chipmaker guided Q3 revenue below consensus and flagged softer data-center orders [1][2].
Sources: [1] Reuters  [2] Finnhub
_Not financial advice. Information only._
```

17.5 Edge cases & how each is handled

CASE	HANDLING
Real move, no news yet	Send the move + “No clear catalyst in the news yet — watching.” Never invent one.
News present but unrelated to the move	The prompt allows the model to say the headlines don't explain it; the refusal line covers this too.
Move extends further after first alert	A second alert only if it crosses a materially larger band; otherwise silent (de-dup).
LLM down / 429	Degrade to raw top headline; back off; the alert still ships.
Stale or single bad print	Freshness + sanity bounds reject it; no alert from a glitch quote.
Injection in a headline	Treated as data; output is user-facing text only; citation + denylist checks bound the blast radius.
Many chats watch the same ticker	Explain once (cache), fan out throttled; audit once.

WHY THIS DESIGN AND NOT “JUST ASK THE LLM WHY NVDA IS DOWN”

A bare LLM prompt would confidently invent a plausible-sounding reason from stale training data — the single most dangerous failure a market bot can have. Grounding the answer in *today's retrieved headlines*, citing them, and refusing when they're thin is the difference between a tool a trader can trust and one that will eventually, confidently, be wrong at the worst moment.